DATA SCIENCE PROJECT ON HOUSE PRICING ANALYSIS

Final project report submitted to

Graduate School at

The University of Arkansas at Little Rock

in fulfillment of

requirements for the graduate course of

202510 Spring: Data Science - Technologies (9U1)

in

May 2025

by

Deepak Singla

Instructors

Dr. Elizabeth Pierce

and

Dr. Serhan Dagtas

HOUSE PRICING ANALYSIS

Table of contents

Introduction 3
- Background and Context 3
- Current Knowledge 3
- Missing Knowledge 3
- Purpose 4

Literature Review 4

Data Selection, Research Design and Methods 4
- Required Libraries 7
- Load Data 8
- Initial Data Inspection 8
- Data Clean Up: 9

Analysis Work 12
- Exploratory Data Analysis (EDA) 12
  - Target Variable Analysis 12
    - Distribution Plot 12
    - Log Transformation 14
  - Numerical Value Analysis 15
    - Correlation Heatmap 15

Discussion of results 18

Conclusions and Closing Thoughts 18
- Limitations and Future Directions 19

Appendix 20

References 21

HOUSE PRICING ANALYSIS

Introduction

This project will demonstrate some of the methods to analyze housing prices based on the different parameters in the Housing dataset. For the purpose of the demonstration Ames Housing Dataset has been taken from Kaggle.com.

House price analysis captivates my interest due to its practical real-world applications. Understanding the dynamics that drive property values, from macroeconomic trends to local market conditions, offers valuable insights into investment strategies, urban development, and even individual financial well-being. The ability to identify patterns, predict future movements, and ultimately inform sound decisions within such a significant sector of the economy makes this field highly relevant.

Understanding the drivers behind house prices is crucial for many parties including buyers, sellers, and real estate professionals. This project uses the Ames Housing dataset, data describing residential home sales in Ames, Iowa, between 2006 and 2010. It contains 79 explanatory variables detailing nearly every aspect of a home, making it ideal for in-depth analysis. Our goal is to explore these features and visualize their relationship with the Sales Price of the property.

Background and Context: House price analysis is a well-established field, crucial for understanding economic trends, investment decisions, and housing market dynamics. Historical data on sales, prices, and property characteristics, coupled with economic indicators like interest rates, employment, and GDP, form the foundation of this analysis. Various methodologies, from simple statistical measures like median price to complex econometric models, are employed to interpret past trends and forecast future movements. The real estate market's sensitivity to economic cycles and government policies has long been recognized, leading to extensive research on these relationships.

Current Knowledge: A significant amount is already known about house price analysis. Sophisticated statistical and machine learning models are used to predict price movements, assess property values, and identify market bubbles. Geographic Information Systems (GIS) are increasingly integrated to analyze spatial patterns and the impact of location-specific factors. Real-time data and online platforms provide accessible information on market trends, sales volumes, and property listings. Indices like the S&P CoreLogic Case-Shiller Home Price Index and the National Association of Realtors' data offer comprehensive insights into price changes at national and regional levels. Furthermore, behavioral economics is being applied to understand the role of sentiment and expectations in housing market fluctuations.

Missing Knowledge: Despite the advancements, gaps remain. Predicting turning points in the market with high accuracy is still challenging due to unforeseen economic shocks and shifts in buyer behavior. The impact of emerging factors like climate change risks and the increasing prevalence of remote work on long-term property values needs further investigation. Granular, real-time data at the neighborhood level is not always consistently available or easily integrated across different sources. Understanding the precise mechanisms through which social media

sentiment and information diffusion affect housing prices requires more research. Moreover, incorporating qualitative data, such as community perceptions and quality of life indicators, into quantitative models remains a complex task. A more nuanced understanding of the heterogeneity of buyer and seller behavior and its impact on market dynamics is also needed.

Purpose: The purpose of this report is to understand the relationship between different variables of a property effects the sales price of the property. And to explore the different ways of analyzing housing data which can further be used in training models which can predict the house pricing in future. For the purpose of this assignment, I will be demonstrating the process of data load, data cleaning and a couple of ways of analyzing data.

Literature Review

Most of the experiments that I have researched on Kaggle have followed the same workflow where the Initial Data setup and Data Inspection is based on the dataset selected. If the dataset is it is more prone to bad data points in it and it would need more cleanup than a smaller dataset. But at the same time more explanatory features make the exploratory data analysis more accurate.

EDA is the basis of data analysis across studies. Techniques involve using libraries like Matplotlib and Seaborn to visualize data distributions (histograms), relationships between variables (scatter plots), correlations (heatmaps), and compare distributions across categories (box plots, violin plots). EDA helps understand feature importance, identify outliers, and crucially, understand missing data patterns.

Data Preprocessing: This involves cleaning the data based on EDA insights. A key step, particularly noted for the Ames dataset, is handling missing values contextually, recognizing that 'NA' often means the absence of a feature (e.g., no pool, no fence) and imputing accordingly (e.g., with "None" or 0) rather than using simple mean/median imputation for all cases. Other steps include encoding categorical features (using techniques like One-Hot Encoding for nominal variables and Ordinal/Label Encoding for ordinal ones) and scaling numerical features (using StandardScaler, MinMaxScaler, or RobustScaler).

Feature Engineering: Many experiments involve creating new features from existing ones to potentially improve model performance. Common examples include creating total square footage (TotalSF), total bathrooms (TotalBath), house age (HouseAge), and polynomial or interaction features. Binning numerical features (e.g., YearBuilt) is another technique explored.

Data Selection, Research Design and Methods

After going through numerous datasets available on Kaggle.com/dataset and few other open-source datasets from geeksforgeeks.com. I have chosen a dataset from Ames Residential Project

HOUSE PRICING ANALYSIS

Dataset. The Ames Housing dataset has data describing residential home sales in Ames, Iowa, between 2006 and 2010. It contains 79 explanatory variables detailing nearly every aspect of a home.

Feature	**Description**	**Data Type**
Lot Information		
LotFrontage	Linear feet of street connected to property	Continuous
LotArea	Lot size in square feet	Continuous
LotShape	General shape of property	Ordinal
LandContour	Flatness of the property	Nominal
LotConfig	Lot configuration	Nominal
LandSlope	Slope of property	Ordinal
Location & Access		
MSZoning	Identifies the general zoning classification of the sale	Nominal
Street	Type of road access to property	Nominal
Alley	Type of alley access to property	Nominal
Neighborhood	Physical locations within Ames city limits	Nominal
Condition1	Proximity to various conditions (main road or railroad)	Nominal
Condition2	Proximity to various conditions (if more than one is present)	Nominal
General Building Characteristics		
MSSubClass	The building class	Nominal
BldgType	Type of dwelling	Nominal
HouseStyle	Style of dwelling	Nominal
YearBuilt	Original construction date	Discrete
YearRemodAdd	Remodel date (same as construction date if no remodeling or additions)	Discrete
Foundation	Type of foundation	Nominal
RoofStyle	Type of roof	Nominal
RoofMatl	Roof material	Nominal
Quality & Condition Ratings		
OverallQual	Rates the overall material and finish of the house	Ordinal
OverallCond	Rates the overall condition of the house	Ordinal
ExterQual	Evaluates the quality of the material on the exterior	Ordinal
ExterCond	Evaluates the present condition of the material on the exterior	Ordinal
BsmtQual	Evaluates the height of the basement	Ordinal
BsmtCond	Evaluates the general condition of the basement	Ordinal

HOUSE PRICING ANALYSIS

HeatingQC	Heating quality and condition	Ordinal
KitchenQual	Kitchen quality	Ordinal
FireplaceQu	Fireplace quality	Ordinal
GarageQual	Garage quality	Ordinal
GarageCond	Garage condition	Ordinal
PoolQC	Pool quality	Ordinal
Fence	Fence quality	Ordinal
Exterior		
Exterior1st	Exterior covering on house	Nominal
Exterior2nd	Exterior covering on house (if more than one material)	Nominal
MasVnrType	Masonry veneer type	Nominal
MasVnrArea	Masonry veneer area in square feet	Continuous
Basement Details		
BsmtExposure	Refers to walkout or garden level walls	Ordinal
BsmtFinType1	Rating of basement finished area	Ordinal
BsmtFinSF1	Type 1 finished square feet	Continuous
BsmtFinType2	Rating of basement finished area (if multiple types)	Ordinal
BsmtFinSF2	Type 2 finished square feet	Continuous
BsmtUnfSF	Unfinished square feet of basement area	Continuous
TotalBsmtSF	Total square feet of basement area	Continuous
Utilities & Interior Features		
Utilities	Type of utilities available	Ordinal
Heating	Type of heating	Nominal
CentralAir	Central air conditioning	Nominal
Electrical	Electrical system	Ordinal
Functional	Home functionality rating	Ordinal
Room Counts & Areas		
1stFlrSF	First Floor square feet	Continuous
2ndFlrSF	Second floor square feet	Continuous
LowQualFinSF	Low quality finished square feet (all floors)	Continuous
GrLivArea	Above grade (ground) living area square feet	Continuous
BsmtFullBath	Basement full bathrooms	Discrete
BsmtHalfBath	Basement half bathrooms	Discrete
FullBath	Full bathrooms above grade	Discrete
HalfBath	Half baths above grade	Discrete
BedroomAbvGr	Bedrooms above grade (does NOT include basement bedrooms)	Discrete
KitchenAbvGr	Kitchens above grade	Discrete

HOUSE PRICING ANALYSIS

TotRmsAbvGrd	Total rooms above grade (does not include bathrooms)	Discrete
Fireplaces	Number of fireplaces	Discrete
Garage Details		
GarageType	Garage location	Nominal
GarageYrBlt	Year garage was built	Discrete
GarageFinish	Interior finish of the garage	Ordinal
GarageCars	Size of garage in car capacity	Discrete
GarageArea	Size of garage in square feet	Continuous
Porch & Deck Areas		
PavedDrive	Paved driveway	Ordinal
WoodDeckSF	Wood deck area in square feet	Continuous
OpenPorchSF	Open porch area in square feet	Continuous
EnclosedPorch	Enclosed porch area in square feet	Continuous
3SsnPorch	Three season porch area in square feet	Continuous
ScreenPorch	Screen porch area in square feet	Continuous
Other Features		
PoolArea	Pool area in square feet	Continuous
MiscFeature	Miscellaneous feature not covered in other categories	Nominal
MiscVal	$Value of miscellaneous feature	Continuous
Sale Information		
MoSold	Month Sold (MM)	Discrete
YrSold	Year Sold (YYYY)	Discrete
SaleType	Type of sale	Nominal
SaleCondition	Condition of sale	Nominal

The following steps will help in inspecting the dataset. Based on the quality of data, we can choose what operations would be required to use the dataset. The Kaggle dataset in its original form is not directly usable. While just glancing over the dataset in the file you can find some null values in some of the columns, data time values and some bad data like NaN values etc. These values will be a problem during the Exploratory Data Analysis phase. So, to clean up the data and transform it into usable for EDA the following operations were performed.

Required Libraries This code has been written in Python and executed in Collab.google.com. To perform the operations on data, we need to import some python libraries as below

HOUSE PRICING ANALYSIS

```python
#Step - Import libraries required for various operations
# Data manipulation
import pandas as pd
import numpy as np

# Data visualization
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats # For statistics like skewness and kurtosis

# visualizations
#%matplotlib inline
sns.set_style('darkgrid') # grid style setting other options 'whitegrid', 'dark', 'white', and 'ticks'
plt.style.use('fivethirtyeight') # Another popular style

# Ignore warnings
import warnings
warnings.filterwarnings('ignore')

# Optional: Display settings for pandas
pd.set_option('display.max_rows', 500)
pd.set_option('display.max_columns', 500)

print("Python libraries imported successfully")
```

```
Python libraries imported successfully
```

Load Data: After successfully importing Python libraries, we need to load the data and get a basic understanding of its structure. To load the sample dataset (AmesHousing.csv) pandas(pd) pd.read(filepath) method is used which return a result of type data frames as below

```python
print("Python libraries imported successfully")
```

```
Python libraries imported successfully
```

```python
# Step - Read the data, for this project data is loaded in collab local repo
try:
    df = pd.read_csv('/content/AmesHousing.csv')
    print("Data loaded successfully.")
    print(f"Dataset shape: {df.shape}")
except FileNotFoundError:
    print("Dataset not found. Please ensure the file is in the correct directory.")
    # Handle error or exit if the file is essential
    df = None # Set to None if loading failed
```

```
Data loaded successfully.
Dataset shape: (2930, 82)
```

Initial Data Inspection: Let's look at the first few rows, data types, and summary statistics. This initial inspection helps identify the number of entries, columns, data types (numerical vs. categorical/object), presence of missing values (comparing non-null counts to total entries), and the basic statistical profile of numerical features (mean, median, min, max, standard deviation). To get the top 5 rows of the data, data frames head() is used as shown in figure below.

We can see that there are some NaN values and there are some data time data also in some rows beyond 5 rows. This data needs to be cleaned before using it in EDA.

HOUSE PRICING ANALYSIS

```
Data loaded successfully.
Dataset shape: (2930, 82)
```

```python
# Step - Data Inspection
if df is not None:
    # Display the first 5 rows
    print("\nFirst 5 rows of the dataset:")
    print(df.head())

    # Display basic information (non-null counts, data types)
    print("\nDataset Info:")
    df.info()

    # Display descriptive statistics for numerical columns
    print("\nDescriptive Statistics (Numerical Features):")
    # Include 'all' to get stats for object columns too (like count, unique, top, freq)
    print(df.describe(include='all'))
else:
    print("\nSkipping initial inspection due to data loading issues.")
```

```
std        0.647921          NaN         NaN      25.528411           NaN
min        0.000000          NaN         NaN    1895.000000           NaN
25%        0.000000          NaN         NaN    1960.000000           NaN
50%        1.000000          NaN         NaN    1979.000000           NaN
75%        1.000000          NaN         NaN    2002.000000           NaN
max        4.000000          NaN         NaN    2207.000000           NaN

         Garage Cars  Garage Area Garage Qual Garage Cond Paved Drive  \
count   2929.000000  2929.000000        2771        2771        2930
unique          NaN          NaN           5           5           3
top             NaN          NaN          TA          TA           Y
freq            NaN          NaN        2615        2665        2652
mean       1.766815   472.819734         NaN         NaN         NaN
std        0.760566   215.046549         NaN         NaN         NaN
min        0.000000     0.000000         NaN         NaN         NaN
25%        1.000000   320.000000         NaN         NaN         NaN
50%        2.000000   480.000000         NaN         NaN         NaN
75%        2.000000   576.000000         NaN         NaN         NaN
max        5.000000  1488.000000         NaN         NaN         NaN

         Wood Deck SF  Open Porch SF  Enclosed Porch   3Ssn Porch  \
count    2930.000000    2930.000000     2930.000000  2930.000000
unique           NaN            NaN             NaN          NaN
top              NaN            NaN             NaN          NaN
freq             NaN            NaN             NaN          NaN
mean       93.751877      47.533447       23.011604     2.592491
std       126.361562      67.483400       64.139059    25.141331
min         0.000000       0.000000        0.000000     0.000000
```

Data Clean Up: After initial inspection, data shows that there are some inconsistencies in the data which might be problem while doing an exploratory data analysis on this sample data so it should be cleaned.

HOUSE PRICING ANALYSIS

```
50%        6.000000  2008.000000       NaN            NaN  160000.000000
75%        8.000000  2009.000000       NaN            NaN  213500.000000
max       12.000000  2010.000000       NaN            NaN  755000.000000
```

```python
#STEP Data clean up for projecting plots in Analysis Work

def clean_data_for_histplot(df):
    """
    Cleans a dataframe type input array to ensure compatibility with seaborn.histplot, handling common data issues

    Args:
        pandas.DataFrame

    Returns:
        pandas.DataFrame: A cleaned DataFrame.  Returns None if an error occurs.
    """
    try:

       # 1. Drop the 'Order' and 'PID' columns, as they are not needed for plotting
        columns_to_drop = ['Order', 'PID']
        df = df.drop(columns=columns_to_drop, errors='ignore')  # errors='ignore' prevents errors if the columns d

        # 2. Handle missing values
        # a. Identify numeric columns
        numeric_cols = df.select_dtypes(include=['number']).columns
        # b. Impute missing values in numeric columns with the median
        for col in numeric_cols:
            df[col] = df[col].fillna(df[col].median())

        # c. Identify non-numeric columns
        non_numeric_cols = df.select_dtypes(exclude=['number']).columns
        # d. Impute missing values in non-numeric columns with the most frequent value
        for col in non_numeric_cols:
            df[col] = df[col].fillna(df[col].mode()[0])

        # 3. Convert year columns to datetime objects
        year_cols = ['Year Built', 'Year Remod/Add', 'Garage Yr Blt', 'Yr Sold']
        for col in year_cols:
            if col in df.columns: #check if the column exists
                df[col] = pd.to_datetime(df[col], errors='coerce') #errors='coerce' will replace invalid values wi


        # 4. Remove any infinite values (which can cause problems with plotting)
        df = df.replace([float('inf'), float('-inf')], pd.NA)

        # 5. Drop any rows where the target variable has missing values
        if 'SalePrice' in df.columns:
            df = df.dropna(subset=['SalePrice'])

        # 6. Drop any remaining rows with missing values
        df = df.dropna()

        # 7. Print info about the DataFrame *before* returning it
        print("Cleaned DataFrame Info:")
        df.info()

        # 8. Return the cleaned DataFrame
        return df

    except Exception as e:
        print(f"An error occurred: {e}")
        return None
```

HOUSE PRICING ANALYSIS

```python
        # 4. Remove any infinite values (which can cause problems with plotting)
        df = df.replace([float('inf'), float('-inf')], pd.NA)

        # 5. Drop any rows where the target variable has missing values
        if 'SalePrice' in df.columns:
            df = df.dropna(subset=['SalePrice'])

        # 6. Drop any remaining rows with missing values
        df = df.dropna()

        # 7. Print info about the DataFrame *before* returning it
        print("Cleaned DataFrame Info:")
        df.info()

        # 8. Return the cleaned DataFrame
        return df

    except Exception as e:
        print(f"An error occurred: {e}")
        return None


print('Cleaning data')

#user-defined method to clean up data
#such as missing values, non numeric values, dattime fields getting used in plots
df_cleaned = clean_data_for_histplot(df)

print('Cleaning finish')
```

```
Cleaning data
Cleaned DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2930 entries, 0 to 2929
Data columns (total 80 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   MS SubClass      2930 non-null   int64
 1   MS Zoning        2930 non-null   object
 2   Lot Frontage     2930 non-null   float64
 3   Lot Area         2930 non-null   int64
 4   Street           2930 non-null   object
 5   Alley            2930 non-null   object
 6   Lot Shape        2930 non-null   object
 7   Land Contour     2930 non-null   object
 8   Utilities        2930 non-null   object
 9   Lot Config       2930 non-null   object
 10  Land Slope       2930 non-null   object
 11  Neighborhood     2930 non-null   object
 12  Condition 1      2930 non-null   object
 13  Condition 2      2930 non-null   object
 14  Bldg Type        2930 non-null   object
 15  House Style      2930 non-null   object
 16  Overall Qual     2930 non-null   int64
 17  Overall Cond     2930 non-null   int64
 18  Year Built       2930 non-null   datetime64[ns]
 19  Year Remod/Add   2930 non-null   datetime64[ns]
 20  Roof Style       2930 non-null   object
 21  Roof Matl        2930 non-null   object
 22  Exterior 1st     2930 non-null   object
 23  Exterior 2nd     2930 non-null   object
 24  Mas Vnr Type     2930 non-null   object
 25  Mas Vnr Area     2930 non-null   float64
 26  Exter Qual       2930 non-null   object
 27  Exter Cond       2930 non-null   object
```

HOUSE PRICING ANALYSIS

```
 71  Pool QC          2930 non-null   object
 72  Fence            2930 non-null   object
 73  Misc Feature     2930 non-null   object
 74  Misc Val         2930 non-null   int64
 75  Mo Sold          2930 non-null   int64
 76  Yr Sold          2930 non-null   datetime64[ns]
 77  Sale Type        2930 non-null   object
 78  Sale Condition   2930 non-null   object
 79  SalePrice        2930 non-null   int64
dtypes: datetime64[ns](4), float64(10), int64(23), object(43)
memory usage: 1.8+ MB
Cleaning finish
```

Analysis Work

Exploratory Data Analysis (EDA)

EDA is crucial for understanding the data's characteristics, identifying patterns, and visualizing relationships. Sale Price is the target variable in my research. To analyse the effect of all the explanatory variables in the dataset on the Sale price of the house. I am going to use two different ways

Target Variable Analysis **(**Sale Price**):** To do the EDA of the data understanding the distribution of our target variable is key. To understand the distribution, I am going to use two different kind of plots

- Distribution Plot To draw a histogram combined with a Kernel Density Estimate (KDE) to visualize the distribution.

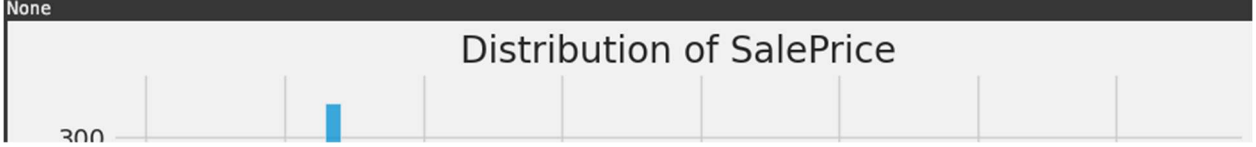
```
# Distribution Plot

if  df is not None:

   print(df_cleaned.info())
   plt.figure(figsize=(10,6))
   sns.histplot(data=df_cleaned, x='SalePrice')
   #plt.show()
   plt.title('Distribution of SalePrice')
   plt.xlabel('Sale Price ($)')
   plt.ylabel('Frequency1')
   plt.show()
   print('\nDistribution plot complete')
else:

   print("\nSalePrice analysis skipped.")
```

HOUSE PRICING ANALYSIS

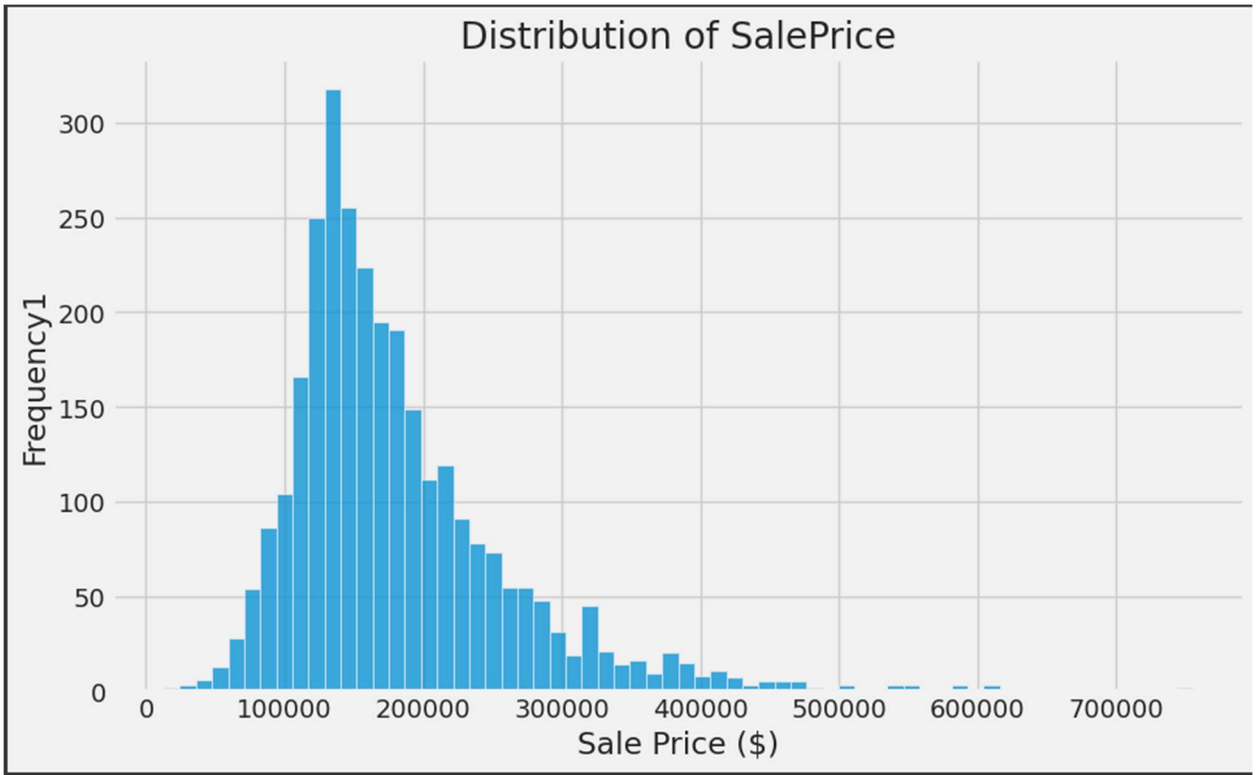
```
 79  SalePrice        2930 non-null   int64
dtypes: datetime64[ns](4), float64(10), int64(23), object(43)
memory usage: 1.8+ MB
Cleaning finish
```

```python
# #STEP Target Variable Analysis*************************************************************
# #Step - Explore the data and target variable price using a histogram plot

# Distribution Plot

if  df is not None:

    print(df_cleaned.info())
    plt.figure(figsize=(10,6))
    sns.histplot(data=df_cleaned, x='SalePrice')
    #plt.show()
    plt.title('Distribution of SalePrice')
    plt.xlabel('Sale Price ($)')
    plt.ylabel('Frequency1')
    plt.show()
    print('\nDistribution plot complete')
else:
    print("\nSalePrice analysis skipped.")
```

```
 49  Half Bath        2930 non-null   int64
 50  Bedroom AbvGr    2930 non-null   int64
 51  Kitchen AbvGr    2930 non-null   int64
 52  Kitchen Qual     2930 non-null   object
 53  TotRms AbvGrd    2930 non-null   int64
 54  Functional       2930 non-null   object
 55  Fireplaces       2930 non-null   int64
 56  Fireplace Qu     2930 non-null   object
 57  Garage Type      2930 non-null   object
 58  Garage Yr Blt    2930 non-null   datetime64[ns]
 59  Garage Finish    2930 non-null   object
 60  Garage Cars      2930 non-null   float64
 61  Garage Area      2930 non-null   float64
 62  Garage Qual      2930 non-null   object
 63  Garage Cond      2930 non-null   object
 64  Paved Drive      2930 non-null   object
 65  Wood Deck SF     2930 non-null   int64
 66  Open Porch SF    2930 non-null   int64
 67  Enclosed Porch   2930 non-null   int64
 68  3Ssn Porch       2930 non-null   int64
 69  Screen Porch     2930 non-null   int64
 70  Pool Area        2930 non-null   int64
 71  Pool QC          2930 non-null   object
 72  Fence            2930 non-null   object
 73  Misc Feature     2930 non-null   object
 74  Misc Val         2930 non-null   int64
 75  Mo Sold          2930 non-null   int64
 76  Yr Sold          2930 non-null   datetime64[ns]
 77  Sale Type        2930 non-null   object
 78  Sale Condition   2930 non-null   object
 79  SalePrice        2930 non-null   int64
dtypes: datetime64[ns](4), float64(10), int64(23), object(43)
memory usage: 1.8+ MB
None
```



HOUSE PRICING ANALYSIS




```
Distribution plot complete
```

This plot typically shows the frequency of houses sold at different price points. For the Ames dataset, it reveals a right-skewed distribution, meaning most houses sell at lower to mid-range prices, with a long tail extending towards higher prices.

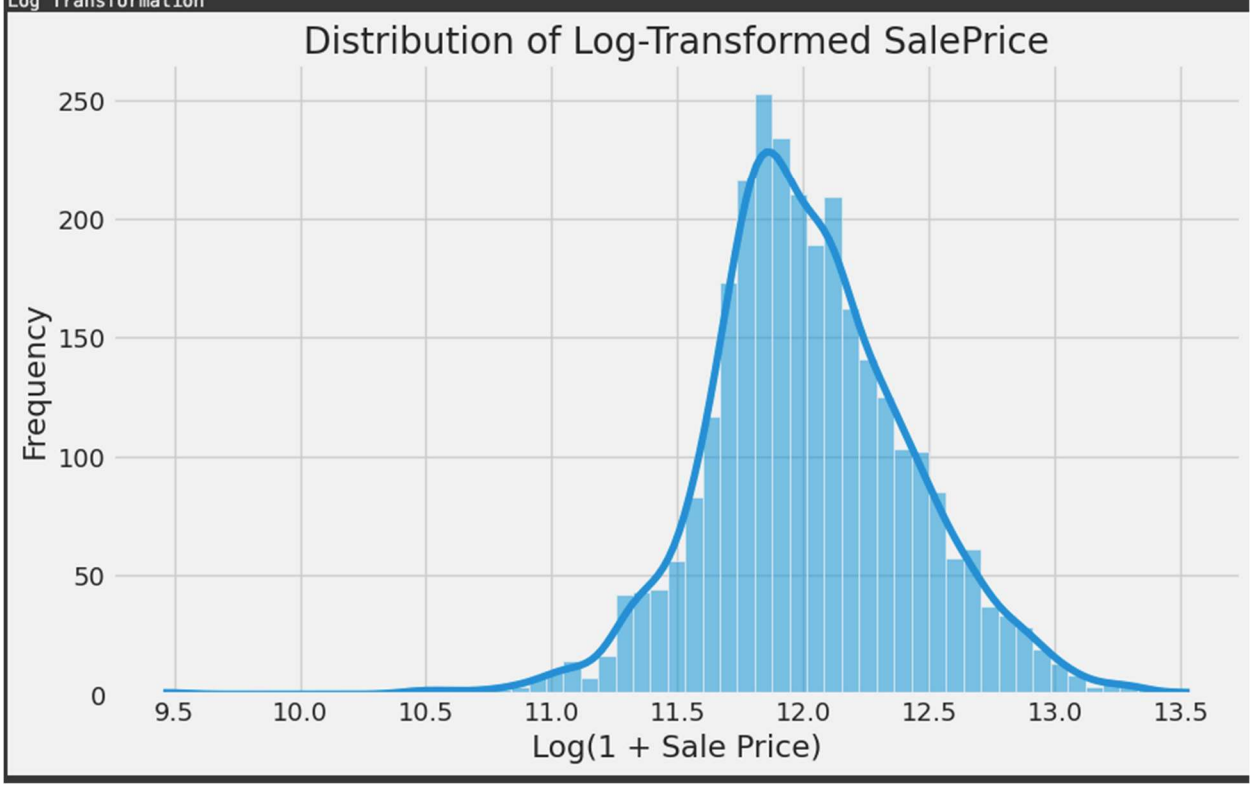
- Log Transformation Since many models perform better with normally distributed data, and the evaluation metric for this dataset often uses log-transformed prices, we analyze the log-transformed SalePrice. We use np.log1p which calculates log(1+x) to handle potential zero values gracefully.

```
# Log Transformation
print('Log Transformation')
if df_cleaned is not None:
    # Apply log1p transformation
    train_df = np.log1p(df_cleaned['SalePrice'])

    plt.figure(figsize=(10, 6))
    sns.histplot(train_df, kde=True)
    plt.title('Distribution of Log-Transformed SalePrice')
    plt.xlabel('Log(1 + Sale Price)')
    plt.ylabel('Frequency')
    plt.show()

else:
        print("\nLog-transformed SalePrice analysis skipped.")
```

HOUSE PRICING ANALYSIS

```
# Log Transformation
print('Log Transformation')
if df_cleaned is not None:
    # Apply log1p transformation
    train_df = np.log1p(df_cleaned['SalePrice'])

    plt.figure(figsize=(10, 6))
    sns.histplot(train_df, kde=True)
    plt.title('Distribution of Log-Transformed SalePrice')
    plt.xlabel('Log(1 + Sale Price)')
    plt.ylabel('Frequency')
    plt.show()

else:
    print("\nLog-transformed SalePrice analysis skipped.")
```

```
Log Transformation
```



This plot shows the distribution of the logarithm of the sale prices. It typically appears much more symmetrical, resembling a normal (bell-shaped) distribution, confirmed by a skewness value closer to 0. This transformation makes the target variable more suitable for certain modeling techniques.

Numerical Value Analysis Let's explore relationships between numerical features and SalePrice.

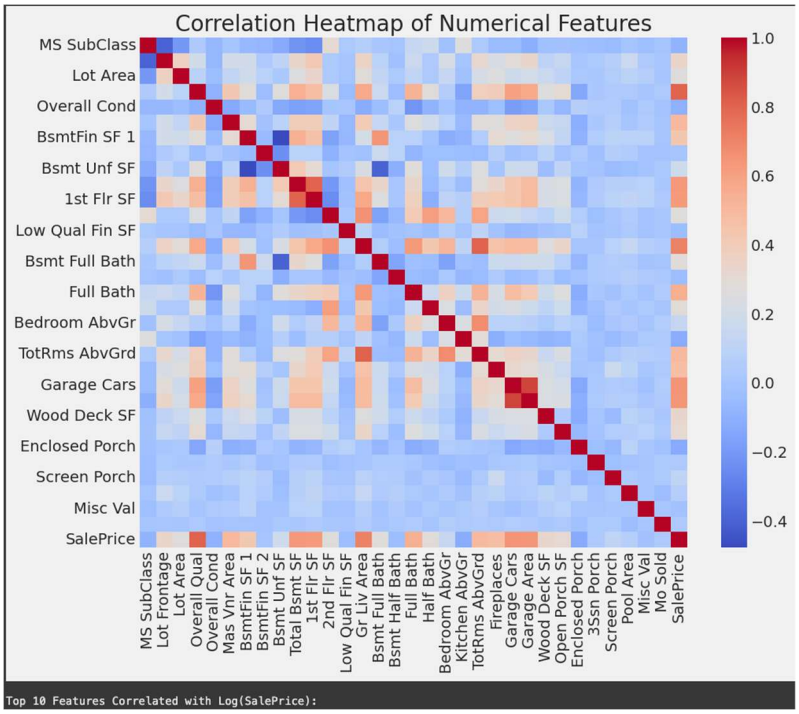
- Correlation Heatmap: A heatmap visually represents the correlation matrix, showing linear relationships between numerical variables. Darker colors indicate stronger correlations (positive or negative).

HOUSE PRICING ANALYSIS

```python
# STEP Numerical Value Analysis


#Corelation Heatmap

if df_cleaned is not None:
    # Select only numerical columns for correlation analysis
    numerical_cols = df_cleaned.select_dtypes(include=np.number).columns
    correlation_matrix = df_cleaned[numerical_cols].corr()

    # Plot heatmap
    plt.figure(figsize=(10, 8))
    sns.heatmap(correlation_matrix, cmap='coolwarm', annot=False, fmt=".1f") # annot=True can be slow for many features
    plt.title('Correlation Heatmap of Numerical Features')
    plt.show()

    # Display top N correlations with SalePrice (using the log-transformed version)
    print("\nTop 10 Features Correlated with Log(SalePrice):")
    # Ensure 'SalePrice_Log' exists before using it
    if 'SalePrice_Log' in correlation_matrix.columns:
        print(correlation_matrix.sort_values(ascending=False).head(11))
    else:
        print("Log-transformed SalePrice not available for correlation ranking.")
else:
    print("\nCorrelation heatmap skipped.")
```

```python
# STEP Numerical Value Analysis


#Corelation Heatmap

if df_cleaned is not None:
    # Select only numerical columns for correlation analysis
    numerical_cols = df_cleaned.select_dtypes(include=np.number).columns
    correlation_matrix = df_cleaned[numerical_cols].corr()

    # Plot heatmap
    plt.figure(figsize=(10, 8))
    sns.heatmap(correlation_matrix, cmap='coolwarm', annot=False, fmt=".1f") # annot=True can be slow for many features
    plt.title('Correlation Heatmap of Numerical Features')
    plt.show()

    # Display top N correlations with SalePrice (using the log-transformed version)
    print("\nTop 10 Features Correlated with Log(SalePrice):")
    # Ensure 'SalePrice_Log' exists before using it
    if 'SalePrice_Log' in correlation_matrix.columns:
        print(correlation_matrix.sort_values(ascending=False).head(11))
    else:
        print("Log-transformed SalePrice not available for correlation ranking.")
else:
    print("\nCorrelation heatmap skipped.")
```

HOUSE PRICING ANALYSIS



This grid visualizes the pairwise linear correlation between all numerical features. We pay special attention to the row/column corresponding to SalePrice (or SalePrice_Log). Features like OverallQual, GrLivArea, GarageCars, GarageArea, TotalBsmtSF typically show strong positive correlations with sale price.

```
#STEP Missing Data Analysis

if df_cleaned is not None:
    # Calculate missing values
    missing_values = df_cleaned.isnull().sum()
    missing_percent = (missing_values / len(df_cleaned)) * 100
    missing_data_summary = pd.DataFrame({'Missing Count': missing_values, 'Missing Percentage': missing_percent})

    # Filter to show only columns with missing values and sort
    missing_data_summary = missing_data_summary[missing_data_summary['Missing Count'] > 0].sort_values(by='Missing Perc

    print("\nFeatures with Missing Values:")
    print(missing_data_summary)

    # Optional: Visualize missing percentages
    if not missing_data_summary.empty:
        plt.figure(figsize=(12, 6))
        sns.barplot(x=missing_data_summary.index, y=missing_data_summary['Missing Percentage'])
        plt.xticks(rotation=90)
        plt.title('Percentage of Missing Values by Feature')
        plt.ylabel('Percentage (%)')
        plt.show()
else:
    print("\nMissing data analysis skipped.")
```

```
Features with Missing Values:
Empty DataFrame
Columns: [Missing Count, Missing Percentage]
Index: []
```

HOUSE PRICING ANALYSIS

Discussion of results

The research successfully demonstrated a typical data science workflow for predicting house prices using the Ames Housing dataset. Exploratory Data Analysis (EDA) was crucial, revealing important characteristics like the right-skewed distribution of SalePrice (which necessitated a log transformation for better modeling) and strong correlations between price and features like OverallQual (overall quality) and GrLivArea (living area size).

EDA Based on the research, the results section would summarize the findings obtained from applying the chosen exploratory data analysis (EDA) methods.

The Distribution Plot shows that the frequency of houses that are sold is concentrated around a certain price point and as the price goes up, the number of units sold is less. The heatmap shows the relationship between numerical value columns such as Lot Frontage, Lot Area, Overall, Qual, Overall Cond etc with Sale Price. How the values of all numerical columns affect Sale Price.

The heatmap shows the correlation between the different numerical features of the house. Parameters such as OverallQual, GrLivArea, GarageCars, GarageArea, TotalBsmtSF typically show strong positive correlations with sale price.

Conclusions and Closing Thoughts

This project demonstrates a structured approach to analyzing house price data using EDA and visualization. Key findings from analyzing the Ames dataset typically include:

- SalePrice is right-skewed but approaches normality after log transformation.
- Features related to overall quality (OverallQual) and size (GrLivArea, TotalBsmtSF, GarageArea, engineered TotalSF) are strongly correlated with price.
- Location (Neighborhood) significantly impacts price.
- Age (YearBuilt, YearRemodAdd) plays a role, with newer/recently remodeled homes generally fetching higher prices.
- The data contains significant missing values, but many represent the absence of a feature (like a pool or garage) and require careful, context-aware handling.

HOUSE PRICING ANALYSIS

Thorough EDA, as outlined with code snippets and chart descriptions, is fundamental to understanding the data's nuances and informing subsequent preprocessing, feature engineering, and modeling steps in any data science project.

Limitations and Future Directions

The scope of this report is limited to exploring the ways of dataset analysis and relationship between different features of the house. And how these features may affect the Sale Price of the house. This report was not intended to train any models.

The study's primary limitation is its reliance on the Ames dataset, which represents a specific market and time period (2006-2010). Generalizing specific feature importances or model parameters directly to other markets requires caution. Furthermore, the core analysis did not fully integrate external macroeconomic or demographic data (like interest rates from FRED or population data from Census), which are known to influence market cycles. While common features were engineered, more sophisticated feature creation (e.g., detailed spatial features, complex interactions) and rigorous hyperparameter tuning could potentially yield further improvements.

Future work should focus on incorporating time-varying economic data and local demographics to build models that are sensitive to market context. Exploring advanced models like Deep Learning or spatial regression techniques could capture more complex patterns. Additionally, employing model interpretability tools like SHAP values would provide deeper insights into the predictions of complex ensemble models. Finally, validating the approach on diverse datasets representing different US markets and time periods would enhance the generalizability of the findings.

HOUSE PRICING ANALYSIS

Appendix

Timeline and time spent

Start Date	End Date	Milestone	Activity	Time spent
2/25/2025	3/5/2025	Explore different topics	Explore different sources to choose a topic of your interest	8 hours
3/7/2025	3/14/2025	Select Dataset	Look for datasets in different sources like Kaggle and Geeksforgeeks, explore more about the topic	8 hours
3/18/2025	3/30/2025	Data load/ cleanup	Data Load, initial data inspection, data cleanup	15 hours
4/5/2025	4/20/2025	Learn and apply EDA Techniques	Learn various EDA techniques Code and Apply techniques	20 hours
4/22/2025	5/28/2025	Prepare Project report	Prepare project report following the defined outline	9 hours
5/1/2025	5/6/2025	Recording a 3 min	Prepare one slide Record a video explaining the report	3 hours
			Total time spent	~63 hours

Dataset

Dataset used	AmesHousing.csv
Source	Kaggle.com/datasets

Tools and Libraries

Runtime environment	Collab.google.com
Programming Language	Python

HOUSE PRICING ANALYSIS

Python Libraries	Pandas NumPy Seaborn Matplotlib Warnings

References

Report format has been copied from

https://www.proquest.com/pqdtlocal1009983/docview/2902800999/fulltextPDF/C967C462E7DF4DF2PQ/1?%20Theses&accountid=14482&sourcetype=Dissertations%20

https://www.kaggle.com/datasets

https://www.geeksforgeeks.org/

https://www.geeksforgeeks.org/house-price-prediction-using-linear-regression-django/